NRG Energy Data Report

Applications of Data Preprocessing,

Transformation, and Visualization in Python

Raine Brookshire

2025-10-02

Abstract

This data task analyzes the Jena Climate dataset, which contains weather measurements recorded every 10 minutes from 2009 to 2016. The analysis focuses on uncovering daily and monthly weather patterns through data preprocessing, aggregation, reshaping, and visualization. Additionally, the top 5 windiest days are identified and presented for detailed comparison. A naive forecasting approach is also applied to demonstrate basic predictive modeling on daily average temperature. Moreover, I compared traditional time series modeling (ARIMA) with machine learning approaches (Random Forest with lag features). ARIMA performed well for short-term predictions but struggled with long-term forecasts due to cumulative error, non-stationarity, and lack of external feature support. By incorporating lag features and time-based predictors, Random Forest improved multi-year forecast accuracy.

NRG Energy

Load Analytics

Table of Contents

1 Data Analysis **1**

1.1 Data Description 1

1.2 Data Preprocessing 2

1.3 Data Aggregation 2

2 Literature **3**

2.1 Augmented Dickey-Fuller (ADF) test 4

3 Methodology **4**

3.1 Testing for Random Walk 4

3.2 Time Series Decomposition 5

4 Results **5**

4.1 Heatmap 6

4.2 Key Metrics 6

4.3 ARIMA Forecast 7

4.4 Random Forest Forecast 7

4.5 Data Trends 8

5 Discussion **8**

6 Reference **9**

1 Data Analysis

Data analysis was crucial to completing my task as I first parsed the `Date Time` column as a datetime object and set it as the index for time series operations. Additional columns for day, month, and year were created to facilitate aggregation. Outliers were handled using standard imputation methods, ensuring continuity and integrity in the time series. Daily and monthly aggregations were performed to compute average temperature, humidity, and wind speed, as well as maximum pressure and minimum dew point (see pg. 3). This data wrangling process prepared the dataset for both modeling and exploratory data analysis.

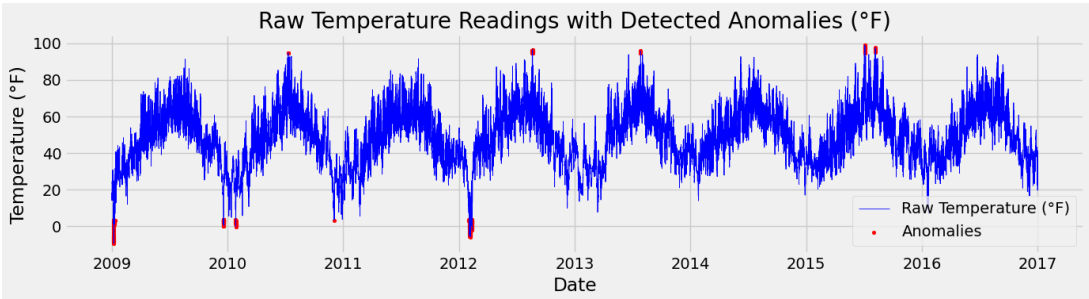

Figure 1: Raw Temperature Data

1.1 Data Description

I utilized the Jena Climate 2009–2016 dataset, which contained 420,552 observations across 15 variables, recorded every 10 minutes. The dataset included key meteorological measurements such as air temperature (T (degC)), dew point (Tdew (degC)), air pressure (p (mbar)), relative humidity (rh (%)), wind speed (wv (m/s)), maximum 10-minute wind (max. wv (m/s)), and other variables capturing atmospheric conditions. My goal was to forecast the daily average temperature using two complementary approaches: traditional time series modeling (ARIMA) to capture temporal trends and seasonality, and forecasting via a random forest approach to leverage past temperature patterns for predictive modeling.

```
import pandas as pd
# Load dataset with Date Time parsed
df = pd.read_csv('jena_climate_2009_2016.csv', parse_dates=['Date Time'])
```

1.2 Data Preprocessing

When I first looked at the provided dataset, I noticed that rows and columns did not contain missing or duplicate values, which is uncommon in real-world data sets. Since the data had already been cleaned, the primary focus of my task involved analyzing and transforming the dataset.

```
# Extract year, month, day, hour, weekday, day of year
df["Year"] = df["Date Time"].dt.year
df["Month"] = df["Date Time"].dt.month
df["Day"] = df["Date Time"].dt.day
df["Hour"] = df["Date Time"].dt.hour
df["DayOfWeek"] = df["Date Time"].dt.day_name()
df["DayOfYear"] = df["Date Time"].dt.dayofyear

print(df.head())
```

1.3 Data Aggregation

I calculated daily and monthly aggregated metrics to better understand trends in the Jena Climate dataset. For daily averages, I computed the mean temperature, humidity, and wind speed, providing a simplified view of day-to-day meteorological conditions. For monthly aggregations, I determined the maximum surface air pressure and the minimum dew point temperature, highlighting the extreme values within each month.

```
# Remove invalid year 2017
df = df[df["Date Time"].dt.year <= 2016]
# Set datetime as index for resampling
df = df.set_index("Date Time")
# Calculate daily averages
daily_avg = df[["T (degC)", "rh (%)", "wv (m/s)"]].resample("D").mean()
print(daily_avg.head())
```

Table 1: Summarixed Daily Average of Temperature, Humidity, and Wind Speed

Date	**T (°C)**	**RH (%)**	**Wind Speed (m/s)**
2009-01-01	-6.81	91.09	0.78
2009-01-02	-3.36	83.51	3.94
2009-01-03	5.44	86.88	2.23
2009-01-04	7.28	61.86	2.84
2009-01-05	12.69	81.56	1.57

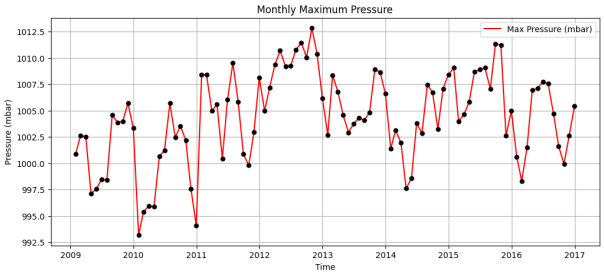

Figure 2: Max Pressure (Min Dew Point in code)

2 Literature

According to Shmueli (2019), forecasting can be seen as an extension of multiple linear regression models and usually includes combined models. A typical multiple linear regression equation can be written as:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p + \varepsilon \tag{1}$$

where:

- $x_1, x_2, \ldots, x_p$ are the predictors
- ε is the error term (noise).

NRG Energy uses forecasting and prediction models to forecast reserves, production, demand, and prices. These companies typically use RMSE, MAE, MAPE, and apply data partitioning/ performance evaluation to maximize results. The methodological framework provided by Shmueli et al. (2019) emphasizes decomposing time series into distinct components—level, trend, seasonality, and noise—which informs the selection and evaluation of appropriate forecasting models.

2.1 Augmented Dickey-Fuller (ADF) test

Random walk testing, which indicates the degree to which the series exhibits predictable behavior, was important to my forecasting model. I checked whether the daily average temperature series was stationary (mean and variance remain constant over time) because models like ARIMA require stationarity for reliable forecasting, I first applied a log transform to stabilize the data. I then used an ARIMA(5,1,0) model after verifying stationarity with the Augmented Dickey-Fuller (ADF) test. In this test, more negative ADF statistics indicate stronger evidence of stationarity, and a p-value below 0.05 allows me to reject the null hypothesis of non-stationarity. I also compared the results against critical values at the 1%, 5%, and 10% levels. A p-value under 0.05 for the residuals confirmed that trends and seasonality were successfully removed, making the series suitable for ARIMA modeling.

3 Methodology

Forecasting typically relies on historical data to predict future values. Approaches range from simple methods, like rolling averages or naive forecasts that assume future values mirror recent trends, to statistical models such as ARIMA that capture seasonality and temporal correlations.

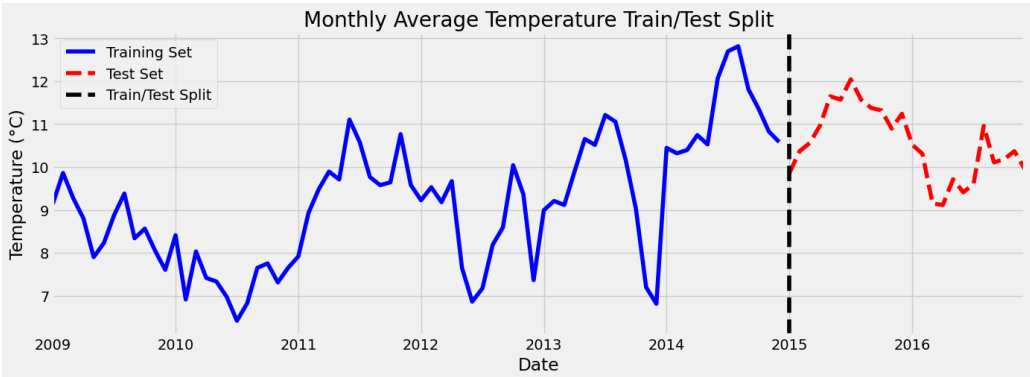

Figure 3

3.1 Testing for Random Walk

The original Jena Climate dataset records measurements every 10 minutes, resulting in 144 data points per day and over 420,000 total records. While this high granularity

captures detailed variability, it also introduces significant noise that can obscure meaningful trends. By resampling to daily averages, I reduce short-term fluctuations such as wind bursts or brief temperature drops, which makes underlying patterns more apparent. Daily aggregation also improves model stability, as predictive models work better with smoother, less volatile data, and reduces computational load, since training on roughly 2,800 daily points is far more efficient than using 420,000 10-minute intervals. This approach complements random walk forecasting, which assumes that tomorrow's value is similar to today's; using daily means reduces the impact of spurious short-term changes, allowing the random walk and other models to capture genuine trends and seasonality more effectively.

3.2 Time Series Decomposition

To better understand the structure of the time series data, it is decomposed into four components: level, which represents the baseline; trend, capturing long-term directional movement; seasonality, reflecting regular periodic fluctuations; and noise, representing short-term unpredictable variation. Decomposition not only aids visualization but also guides model selection, indicating whether AR, ARIMA, or seasonal ARIMA models are appropriate for capturing the observed dynamics (see pg.9).

4 Results

The time series decomposition (figure 7) showed additive seasonality with consistent annual fluctuations of constant amplitude throughout the period, and an increasing trend that rises from approximately 3.20 in 2009 to peak near 3.30 in 2014-2015, representing a warming pattern of about 0.10 units on the log scale. The residuals are generally small and randomly distributed around zero, with one notable outlier in 2012-2013, indicating the decomposition effectively captures the data's structure.

The time series decomposition on the last page shows additive seasonality with consistent annual fluctuations of constant amplitude throughout the period, and an increasing trend that rises from approximately 3.20 in 2009 to peak near 3.30 in 2014-2015, representing a warming pattern of about 0.10 units on the log scale. The residuals are generally small and randomly distributed around zero, with one notable outlier in 2012-2013, indi-

cating the decomposition effectively captures the data's structure.

Wind analysis identified the top 5 windiest days, with the most extreme event occurring on November 23, 2009 (7.64 m/s). Four of these five windiest days were concentrated in winter months (November-February)

Code for metrics for those days in a table where the variables are organized is shown below (Optional Challenge):

```
# --- Top 5 by daily average wind ---
daily_avg_wind = jena_df['wv (m/s)'].resample('D').mean()
top5_avg_days = daily_avg_wind.sort_values(ascending=False).head(5)
top5_avg_dates = top5_avg_days.index

# Define aggregation for all columns
agg_dict = {col: ['mean','min','max'] for col in jena_df.columns}
# Aggregate metrics
daily_stats = jena_df.resample('D').agg(agg_dict)
daily_stats.columns = ['_'.join(col).strip() for col in daily_stats.columns.values]
```

4.1 Heatmap

The heatmap (figure 4) of monthly average temperatures clearly shows that summer months are consistently hotter than other months, reflecting expected seasonal patterns. Additionally, the color intensity gradually increases across the years, suggesting that average temperatures are rising over time, which may be indicative of long-term warming trends or potential effects of global climate change.

4.2 Key Metrics

This table identifies the windiest days based on daily average wind speed, which reflects sustained wind conditions. If maximum wind speed were used instead, the ranking could change, highlighting days with short-lived gusts rather than overall windy conditions. Using max wind emphasizes peak events, which may overstate the impact of brief extreme winds, while daily averages provide a better sense of the overall windy experience throughout the day.

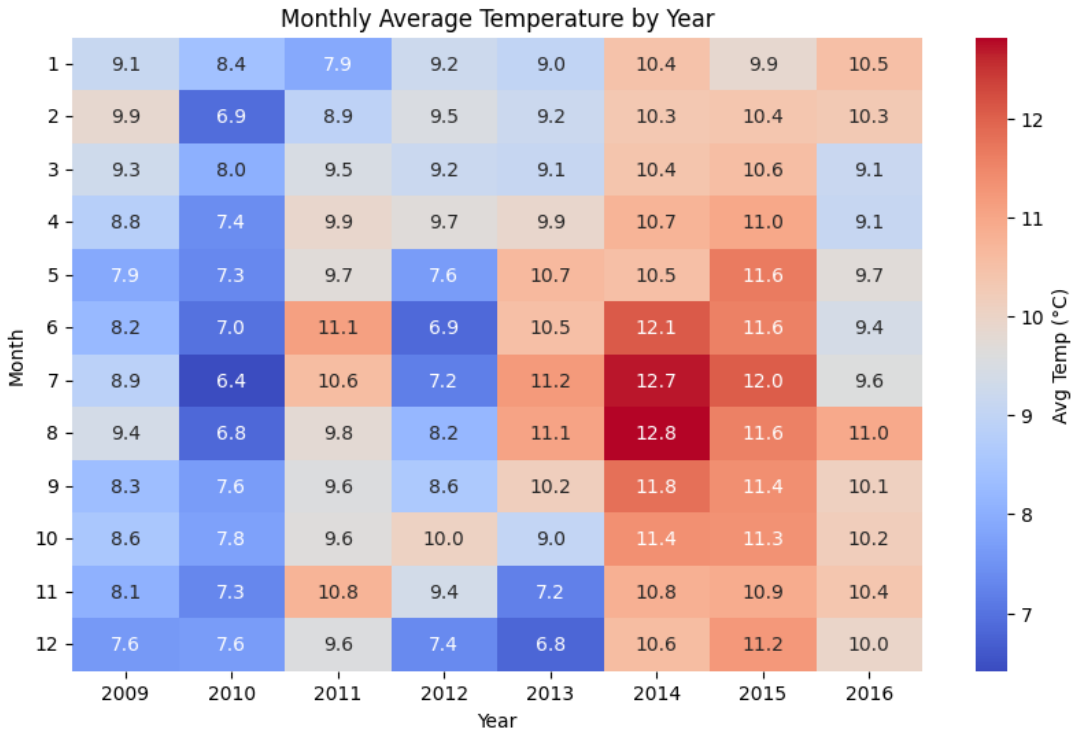

Figure 4: June, July, August usually experienced the highest temperatures

Table 2: Top 5 Windiest Days Based on Daily Average Wind Speed

Date	Mean Wind (m/s)
2009-11-23	7.64
2014-12-12	7.14
2015-11-29	7.08
2016-02-08	6.92
2015-01-13	6.88

4.3 ARIMA Forecast

ARIMA models were fitted to the time series to account for trend and autocorrelation in the data. Differencing and moving average terms were incorporated to improve forecast accuracy and capture both short-term fluctuations and long-term patterns.

4.4 Random Forest Forecast

Using the Random Forest model with lag features, I achieved strong forecasting performance. The model produced a mean absolute error (MAE) of 2.03°C, indicating that predicted temperatures were, on average, within 2°C of the true values. The root mean squared error (RMSE) of 2.54°C shows that larger deviations were rare, and the R^2 score of 0.89 demonstrates that the model explains 89% of the variance in daily temperatures.

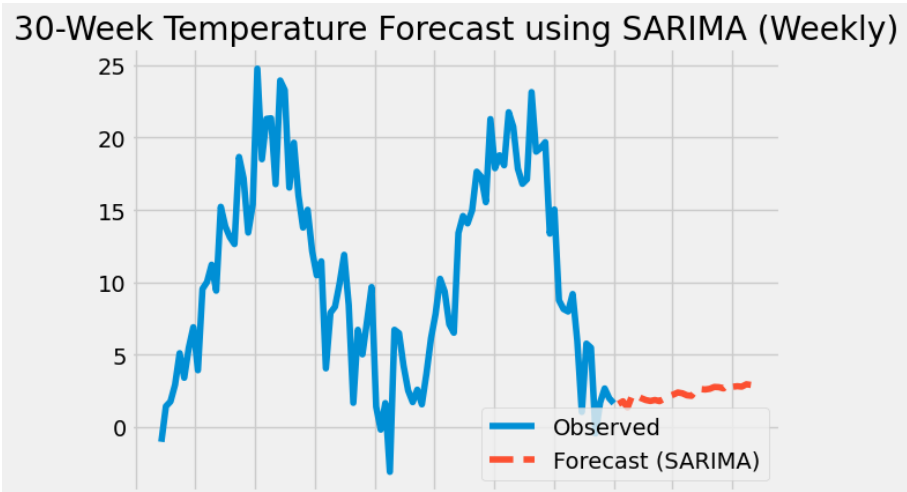

Figure 5: SARIMA model from 2015 to 2016. Forecasts 30 weeks after 2016.

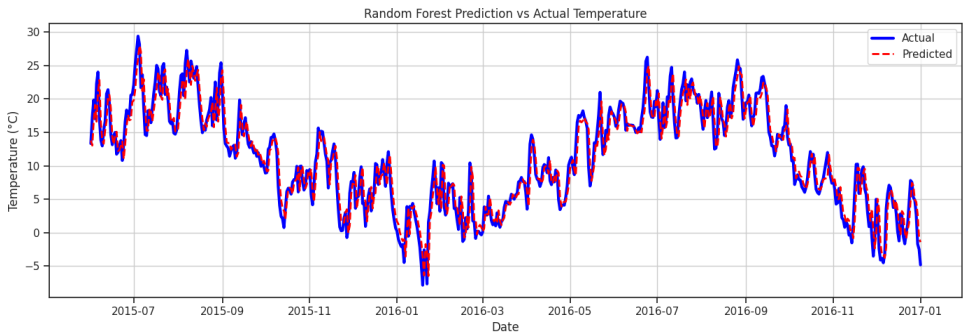

Figure 6: Random Forest Prediction

4.5 Data Trends

The figure above illustrates the monthly trends in temperature across the years. Seasonal patterns are clearly visible, with warmer periods concentrated in summer months and cooler periods in winter. The visualization highlights both short-term fluctuations and long-term trends, providing insights that inform forecasting models.

5 Discussion

My findings were contextualized from the methodological framework of Shmueli et al. (2019), emphasizing the potential application of ARIMA and ADF testing. This textbook also furthered my understanding of the decomposition into level, trend, seasonality, and noise. In my analysis, I found that ARIMA models are effective for short-term temperature forecasting but face significant limitations over longer periods. ARIMA relies on recursive forecasting, predicting day + 1 and then using that prediction to forecast day + 2, which

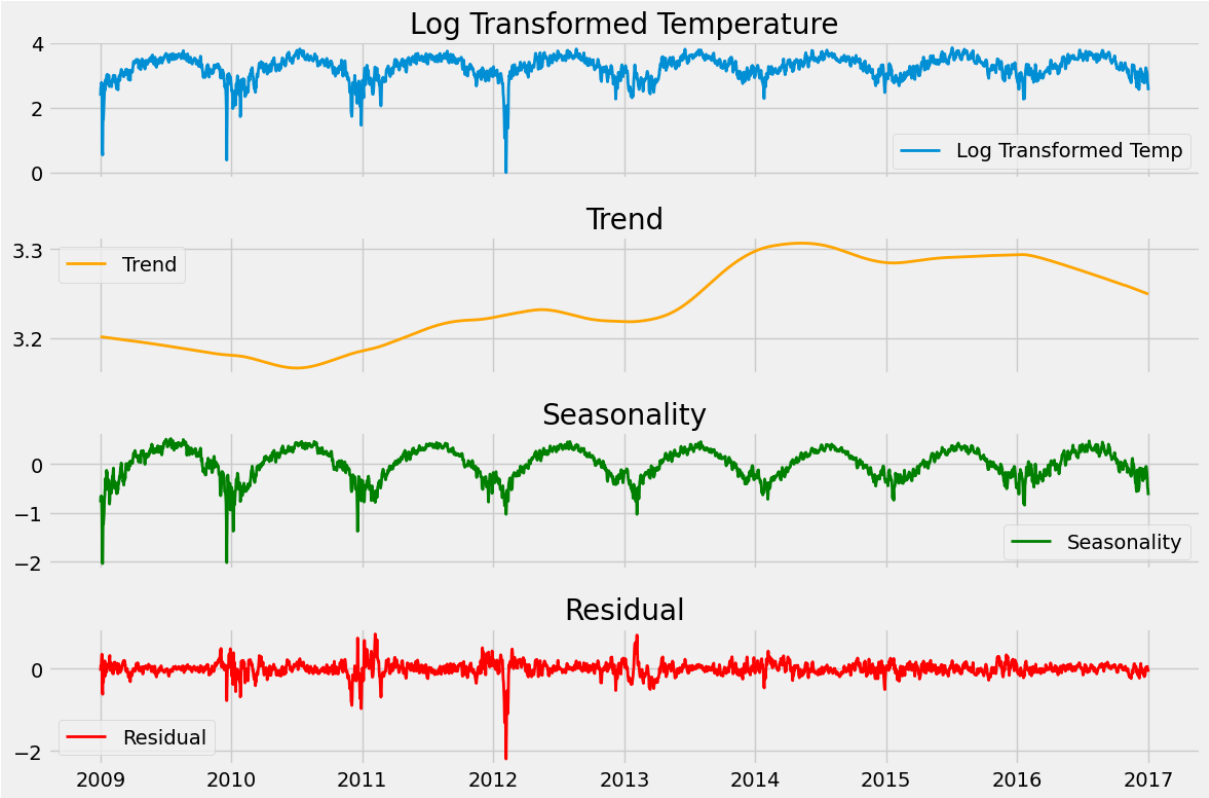

Figure 7: Enter Caption

causes errors to compound.

To address these limitations, I implemented a Random Forest model with lag features, which allowed me to leverage past temperature patterns and include relevant time-based features like day of year, month, and hour. Unlike ARIMA, Random Forest can incorporate multiple predictors and is less sensitive to the stationarity assumption, which helped reduce cumulative errors and improve multi-year forecast accuracy.

6 Reference

Shmueli, G., Bruce, P. C., Gedeck, P., & Patel, N. R. (2019). *Data Mining for Business Analytics: Concepts, Techniques, and Applications in Python*. Wiley.